INF122-style Practice Exam

Java Programming - written exam

Course code	JAVA122
Assessment form	Written practice exam
Duration	3 hours
Total points	100
Allowed resources	No internet. Standard Java API summary may be used if permitted by your teacher.

Instructions

Answer all tasks. Unless otherwise stated, you may assume that inputs are non-null. Write Java code that would compile in a normal Java 17 environment. You do not need to include package declarations.

This exam follows the same broad format as the uploaded exam: multiple-choice value/type questions, then larger programming tasks on I/O, list processing, maps/custom classes, and inductive data structures.

1 Values and expressions in Java

Max points: 12

---

1. What is printed by the following code?

```
System.out.println("abc".substring(1));
```

[ ] abc
[ ] bc
[ ] ab
[ ] Throws StringIndexOutOfBoundsException

2. What is the value of x after this code?

```
int x = 7 / 2;
```

[ ] 3
[ ] 3.5
[ ] 4
[ ] The code gives a type error

3. What is printed?

```
int[] a = {1, 2, 3};
System.out.println(a[1]);
```

[ ] 1
[ ] 2
[ ] 3
[ ] Throws ArrayIndexOutOfBoundsException

4. What is the value of b?

```
boolean b = true || (10 / 0 == 0);
```

[ ] true
[ ] false
[ ] Throws ArithmeticException
[ ] The code gives a type error

5. What is printed?

```
String s = "Hi";
s.toLowerCase();
System.out.println(s);
```

[ ] hi
[ ] HI
[ ] Hi
[ ] The code gives a type error

6. What is printed?

```
ArrayList<Integer> xs = new ArrayList<>(List.of(1, 2, 3));
xs.add(4);
System.out.println(xs.size());
```

[ ] 3
[ ] 4
[ ] [1, 2, 3, 4]
[ ] Throws UnsupportedOperationException

2 Types, classes and generics in Java

Max points: 12

1. Which declaration is valid Java?

[ ] List xs = new ArrayList<>();
[ ] List xs = new ArrayList<>();
[ ] ArrayList xs = new ArrayList();
[ ] Map m = new HashMap<>();

2. Which method header correctly describes a method that takes a String and returns its length?

[ ] int length(String s)
[ ] String length(int s)
[ ] void length(String s)
[ ] int length = String s

3. Which statement about String in Java is correct?

[ ] String objects are mutable.
[ ] String objects are immutable.
[ ] Calling toUpperCase changes the original String in place.
[ ] String cannot be compared with equals.

4. Given class Dog extends Animal, which assignment is valid?

[ ] Dog d = new Animal();
[ ] Animal a = new Dog();
[ ] ArrayList xs = new ArrayList();
[ ] Dog d = (Dog) new Object(); is always safe.

5. What is the most precise return type for a method that might fail to find a Student?

[ ] Student
[ ] null only
[ ] Optional
[ ] void

6. What does static mean in a method declaration?

[ ] The method belongs to the class rather than one object.
[ ] The method can never be called.
[ ] The method cannot use parameters.
[ ] The method is automatically private.

3 Console I/O and string processing

Max points: 14

In this task you show that you understand basic console input/output, loops, and string processing in Java.

You may use Scanner, StringBuilder, Character.toUpperCase and Character.toLowerCase.

a) [6 points] Write a method readSentence that reads characters from standard input until the user enters a period character ".". The returned String must include the period.

```
public static String readSentence(Scanner sc) {
    // TODO
}
```

b) [8 points] Write a main method that uses readSentence and prints the sentence with the word order reversed. The first word in the result must start with an uppercase letter, and the last word must start with a lowercase letter. You may assume at least one word and no proper nouns.

```
Input:  This is a test.
Output: Test a is this.
```

4 Arrays, ArrayList and higher-order style operations

Max points: 25

In this task you show that you understand list processing in Java. You may use loops. Where the task explicitly asks for streams, use java.util.stream.

```
import java.util.*;
import java.util.stream.*;
```

a) [5 points] Write a method squareEvens that returns a new list containing the square of every even number in the input list. Example: squareEvens(List.of(1,2,3,4)) returns [4, 16].

```
public static List<Integer> squareEvens(List<Integer> xs) {
    // TODO
}
```

b) [5 points] Write the same function as in a), but this time use stream(), filter and map instead of an explicit loop.

```
public static List<Integer> squareEvensStream(List<Integer> xs) {
    // TODO
}
```

c) [5 points] Write a generic method countNull that counts how many elements in a list are null. The method must work for lists of any element type.

```
public static <T> int countNull(List<T> xs) {
    // TODO
}
```

d) [5 points] Write a method dedupeConsecutive that removes consecutive duplicates, but keeps one copy of each run. Example: [1,1,2,2,2,3,1,1] becomes [1,2,3,1].

```
public static <T> List<T> dedupeConsecutive(List<T> xs) {
    // TODO
}
```

e) [5 points] Let duplicateEach(xs) return a list where every element occurs twice in a row. Is dedupeConsecutive(duplicateEach(xs)).equals(xs) true for every List xs? Give a precise explanation.

```
public static <T> List<T> duplicateEach(List<T> xs) {
```

```
    // imagine this is already implemented
}
```

5 HashMap and custom classes

Max points: 20

In this task you implement a small inventory and checkout model for a shop. The item id identifies a product type.

```
import java.util.*;

class ItemInfo {
    String name;
    int price; // kroner
    ItemInfo(String name, int price) {
        this.name = name;
        this.price = price;
    }
}

enum OfferType { PERCENT, THREE_FOR_TWO }

class Offer {
    OfferType type;
    int percent; // only used when type == PERCENT
    Offer(OfferType type, int percent) {
        this.type = type;
        this.percent = percent;
    }
}

interface BillItem {}
record Purchase(int itemId, ItemInfo info) implements BillItem {}
record Discount(int amount) implements BillItem {}

class Shop {
    Map<Integer, Integer> inventory = new HashMap<>(); // itemId -> amount
    Map<Integer, ItemInfo> products = new HashMap<>(); // itemId -> info
    Map<Integer, Offer> offers = new HashMap<>();      // itemId -> offer
    int till;
}
```

Example: If bananas cost 15 kr and have a three-for-two offer, buying 5 bananas gives one free banana: 5 * 15 - 15 = 60 kr.

a) [6 points] Write computePercentDiscount. It must compute the discount amount rounded to the nearest whole krone. Example: computePercentDiscount(30, 201) == 60. Use double for intermediate calculations.

```
public static int computePercentDiscount(int percent, int price) {
    // TODO
}
```

b) [6 points] Write takeItem. It removes one item from the inventory only if there is at least one item of that type. If the amount becomes 0, remove the key from the map.

```
public static void takeItem(int itemId, Map<Integer, Integer> inventory) {
    // TODO
}
```

c) [8 points] Write addItem. If the product exists, add a Purchase to the bill. Then add a Discount when a valid offer applies. For THREE_FOR_TWO, look at how many purchases of this item will be in the bill after the current item is added; every third item gives a discount equal to the item price.

```
public static void addItem(Shop shop, int itemId, List<BillItem> bill) {
    // TODO
}
```

6 Inductive data structures: word trie

Max points: 17

In this task you show that you understand recursive data structures. We represent a set of words as a trie. A node marks whether a word may end here, and maps each possible next character to a subtree.

```
import java.util.*;

class WordTree {
    boolean endAllowed;
    Map<Character, WordTree> continuations;

    WordTree(boolean endAllowed) {
        this.endAllowed = endAllowed;
        this.continuations = new HashMap<>();
    }
}
```

The invariant is that a character has at most one continuation from each node. Using Map enforces this at the level of keys.

a) [7 points] Write insertWord. It must add the given word as a possible word in the tree. The method may mutate the tree.

```
public static void insertWord(String word, WordTree tree) {
    // TODO
}
```

b) [4 points] Explain why your insertWord method preserves the invariant that each character has at most one continuation from a node.

c) [6 points] Write allWords, which returns all words represented by the tree. The order of the returned list does not matter.

```
public static List<String> allWords(WordTree tree) {
    // TODO
}

// You may write a private recursive helper method if you want.
```

Point summary

Task	Topic	Points
1	Values and expressions	12
2	Types, classes and generics	12
3	Console I/O and string processing	14
4	Arrays, ArrayList and higher-order style operations	25
5	HashMap and custom classes	20
6	Inductive data structures: word trie	17
	Total	**100**

End of exam.